MAKSDEV STUDIO

ENGLISH EDITION / PYTHON 3

# Python From First Line to Real Projects

Understand the Logic.
Write the Code.
Build Something Useful.



31 LESSONS & SECTIONS · 4 PROJECTS · 100 CHALLENGES

**Md. Makshedul Islam**

Student, EXIM Bank Agricultural University Bangladesh.

# Your First Step

You do not need to memorize Python before you can use it. You need a small set of reliable ideas: values represent information, expressions calculate results, decisions choose a path, loops repeat work, and functions give that work a name.

This book builds those ideas in that order. You will predict what a program does, trace its changing values, run it, and then improve it. The purpose is not to type someone else's answer. It is to understand why the answer works and when it does not.

The examples use Python 3.10 or newer. Most work on older Python 3 releases, but `match` and annotations such as `int | str` require Python 3.10+. Install a supported Python 3 version from python.org. This is a learning path, not a promise that a particular third-party service or package will stay unchanged.

## How to study a lesson

1. Read the problem before the code.
2. Write down the input and the required output.
3. Predict the result of the example.
4. Run the accompanying script and compare.
5. Change one thing and predict again.
6. Complete the practice without looking at the solution first.

Code blocks labeled Python are also supplied as numbered `.py` files. The example index in the companion folder maps filenames to lessons. Output blocks show the expected console result; prompts in interactive examples are part of the program. Terminal commands are not Python code.

## A notebook habit

For each exercise, keep four short notes: **my plan, my prediction, my result, my explanation**. A mismatch between prediction and result is useful evidence. Follow it rather than hiding it.

# Contents

A COMPLETE LEARNING PATH

1 · Programs Are Precise Instructions ..... 4
2 · Install Python and Use the Terminal ..... 6
3 · Values, Variables and Types ..... 8
4 · Input, Output and Conversion ..... 10
5 · Operators and Expressions ..... 12
6 · Decisions and Boolean Logic ..... 14
7 · Repetition with While ..... 16
8 · Iteration with For and Range ..... 18
9 · Tracing, Nested Loops and Patterns ..... 20
10 · Strings and Text Processing ..... 22
11 · Lists and Mutable Objects ..... 24
12 · Tuples and Unpacking ..... 26
13 · Dictionaries and Records ..... 27
14 · Sets and Membership ..... 29
15 · Functions, Arguments and Return Values ..... 31
16 · Scope, Recursion and Problem Decomposition ..... 33
17 · Modules, Packages and Virtual Environments ..... 35
18 · Files, Paths, CSV and JSON ..... 37
19 · Exceptions, Validation and Debugging ..... 39
20 · Testing and Program Design ..... 41
21 · Classes and Object-Oriented Thinking ..... 43
22 · Inheritance, Properties and Special Methods ..... 45
23 · Comprehensions, Iterators and Functional Tools ..... 47
24 · Type Hints, Match and Assignment Expressions ..... 49
25 · Project: A Resilient Calculator ..... 51
26 · Project: A Quiz with a Score Report ..... 53
27 · Project: An Expense Tracker with JSON Persistence ..... 55
28 · Project: A Safe Assistant and Reply-Draft Workflow ..... 59
29 · One Hundred Practice Challenges ..... 61
30 · Selected Worked Solutions ..... 65
31 · Quick Reference, Glossary and Next Steps ..... 72

# 1 · Programs Are Precise Instructions




01 / A value enters, a rule transforms it, and a result leaves.

## Problem

A shop has three notebooks at 45 taka each. How much must a customer pay?

A human may answer immediately. A program needs the quantities and the rule connecting them. The useful plan is **read quantity and price → multiply → display total**. This is an algorithm: a finite, ordered method for solving a problem.

An algorithm is not tied to a language. You could write this plan in English, draw it, or implement it in Python. Coding begins after you understand the rule.

```
quantity = 3
price = 45
total = quantity * price
print(f"Total: {total} taka")
```

### Expected output

```
Total: 135 taka
```

`quantity = 3` binds the name `quantity` to an integer value. The next line supplies the unit price. Python evaluates the expression on the right of `=` before assigning its result to `total`. Finally, `print()` displays text. It does not collect money or operate a physical device.

## The input–process–output model

| Stage | Question | Notebook example |
|---|---|---|
| Input | What information do I need? | Quantity and unit price |
| Process | What rule transforms it? | Multiplication |
| Output | What should the user see? | Total cost |

## Flowchart language

In this book, an oval means start/end, a parallelogram means input/output, a rectangle means processing, and a diamond means a decision. Arrows show the direction of control. A loop has an arrow that returns to an earlier decision.

**Flow:** Start → obtain quantity and price → calculate total → display total → End.

## Common mistakes

- Coding before deciding what the result should mean.
- Forgetting the unit: 135 taka and 135 notebooks are different quantities.
- Thinking a displayed instruction performs the action it describes.

## Practice

Plan the area of a rectangle without writing code. State its inputs, formula and output. Then choose a test case whose answer you can calculate by hand.

# 2 · Install Python and Use the Terminal

## Two places to type

The **terminal** runs operating-system commands. The **Python interpreter** executes Python expressions and statements. A terminal prompt might end in `$` or `>`. Python's interactive prompt is `>>>`. Do not type those prompt symbols as part of a command.

On Windows, the Python launcher may be available as `py`; on macOS and Linux, `python3` is common. The commands below use `python`. If that command is missing or points to something else, substitute the command that launches your Python 3 installation.

```
python --version
python -m pip --version
```

If Python is not found, install it and follow the installer's PATH guidance. On Linux, use your distribution's package instructions rather than replacing its system Python. An editor such as VS Code is optional; a plain text editor is enough for the first examples.

## Your first file

Create a folder named `python-work`. Save the following as `hello.py`, not `hello.py.txt`.

```python
```

# A comment explains intent; it is not executed.

name = "Python learner" print(f"Hello, {name}!") print("A small program is still a real program.")

> From a terminal opened in that folder:
>
> ```sh
> python hello.py

The file is the input to the interpreter. Saving a file and running a file are separate steps. If you edit without saving, the interpreter may run the previous version.

## The REPL

Run `python` without a filename to enter the interactive read–evaluate–print loop. Try `2 + 3`, then `type(2)`. Use `exit()` to leave. In a script, use `print(2 + 3)` when you want to see a result; a bare expression is not automatically printed as it is in the REPL.

## Indentation is structure

Python uses indentation to group statements. Four spaces per level is the standard convention. Do not mix tabs and spaces. Extra spaces inside ordinary expressions may be harmless, but indentation and some line breaks are meaningful.

## Troubleshooting checklist

- **File not found:** check the terminal's current folder and the filename.
- **Syntax error:** read the line number and the preceding line; a missing bracket may be earlier.
- **No output:** confirm that the file contains a `print()` or calls the intended function.
- **Wrong interpreter:** compare `python --version` with the editor's selected interpreter.

## Practice

Print three lines: your name, one thing you want to automate, and the answer to `12 * 12`. Change the file, save it, then run it again.

# 3 · Values, Variables and Types

## Mental model

A value is a piece of data. A type determines the operations that make sense for it. A variable is a name bound to an object; it is not a permanently typed box.

| Type | Example | Typical purpose |
|---|---|---|
| `int` | `42` | Counts, whole numbers |
| `float` | `2.5` | Approximate real-number calculations |
| `str` | `"Dhaka"` | Text |
| `bool` | `True` | Yes/no conditions |
| `NoneType` | `None` | No value or not yet supplied |

```
students = 28
average = 76.5
course = "Python"
is_open = True
result = None
print(type(students).__name__)
print(course, is_open, result)
students = students + 1
print(students)
```

### Expected output

```
int
Python True None
29
```

The right side of `students = students + 1` uses the old value, 28. The name is then rebound to 29. Assignment is not an equation to solve. `x = x + 1` is a valid instruction, not a mathematical contradiction.

## Naming well

Use names such as `student_count`, `unit_price` and `is_valid`. Names are case-sensitive: `total` and `Total` differ. A name cannot begin with a digit or be a reserved keyword such as `class`. Avoid replacing built-in names such as `list`, `str`, `sum` or `input`.

## Equality versus identity

`==` asks whether values compare equal. `is` asks whether two references identify the same object. Use `value is None` to check for the sentinel `None`. Do not use `is` to compare ordinary strings or numbers.

## Floating-point reality

Binary floating-point cannot represent every decimal fraction exactly. A calculation such as `0.1 + 0.2` may not equal `0.3` exactly. For measurements, use a tolerance when comparing. For money, this book's expense tracker stores integer minor units instead of repeatedly adding floats.

## Practice

Choose types for a phone number, a student's name, an attendance count and a pass/fail result. A phone number should normally be a string: you do not add phone numbers, and leading zeros matter.

# 4 · Input, Output and Conversion

## Problem

Ask for a learner's name and age, then report their age next year.

`input()` always returns a string. Even if a user types `19`, the result is text until you convert it. A prompt helps the user understand what to enter.

```
name = input("Name: ").strip()
age_text = input("Age in whole years: ")
age = int(age_text)
print(f"Hello, {name}. Next year you will be {age + 1}.")
```

With inputs `Amina` and `19`, the final line is:

```
Hello, Amina. Next year you will be 20.
```

## Line-by-line logic

1. Obtain text and remove surrounding whitespace from the name.
2. Obtain the age as text.
3. Ask `int()` to parse a whole number.
4. Add 1 to the integer and insert it into an f-string.

If the user types `nineteen`, conversion raises `ValueError`. That is a predictable input problem, not evidence that Python is broken. Chapter 19 shows how to recover and ask again.

## Conversion is not magic

`int("12")` works. `int("12.5")` does not, because that text is not an integer representation. `int(12.9)` returns 12 by truncating toward zero; it does not round to the nearest whole number. `str(12)` makes text. `float("12.5")` makes an approximate numeric value.

## Output tools

`print(a, b)` separates values with a space by default. `sep=" | "` changes the separator. `end=""` prevents the usual newline. An f-string can format a number: `f"{price:.2f}"` displays two digits after the decimal point. Formatting changes the presentation, not the underlying arithmetic.

## Common mistakes

- `input() + 1`: text cannot be added to an integer.

- `"10" + "5"`: this joins strings into `"105"`; it does not calculate 15.
- `bool("False")`: this is `True`, because a non-empty string is truthy.
- Expecting `.strip()` to remove spaces inside a name; it removes them at the ends.

## Practice

Ask for length and width, convert both with `float()`, then display their product. Write down what should happen for empty input, negative dimensions and non-numeric input. Implement validation after Chapter 19.

# 5 · Operators and Expressions

## Choose the right rule

| Operator | Meaning | Example result |
|---|---|---|
| `+`, `-`, `*` | Add, subtract, multiply | `7 * 3` → `21` |
| `/` | True division | `7 / 2` → `3.5` |
| `//` | Floor division | `7 // 2` → `3` |
| `%` | Remainder | `7 % 2` → `1` |
| `**` | Exponentiation | `2 ** 3` → `8` |
| `==`, `!=` | Equal, not equal | `3 != 4` → `True` |
| `<`, `<=`, `>`, `>=` | Ordered comparison | `5 >= 5` → `True` |

## Problem: split minutes into hours and minutes

```
total_minutes = 135
hours = total_minutes // 60
minutes = total_minutes % 60
print(f"{hours} hours, {minutes} minutes")
print(-7 // 2)
print(2 + 3 * 4)
print((2 + 3) * 4)
```

### Expected output

```
2 hours, 15 minutes
-4
14
20
```

Floor division rounds down toward negative infinity, not toward zero. That explains `-7 // 2 == -4`. For non-negative durations, `//` and `%` conveniently produce complete hours and leftover minutes.

## Precedence

Multiplication binds more tightly than addition. Parentheses make the intended order explicit. Prefer readable expressions over reliance on memorized precedence rules. Exponentiation is unusual: `2 ** 3 ** 2` means `2 ** (3 ** 2)`.

## Augmented assignment

`score += 5` updates a score. For immutable numbers this rebinds the name to a new result. For mutable collections, some augmented operations can modify the existing object, so do not assume all `+=` operations have identical aliasing behavior.

## Bitwise operators

`&`, `|`, `^`, `~`, `<<` and `>>` operate on integer bits. They are not replacements for Boolean `and`, `or` and `not`. You can postpone bit manipulation until a project needs masks or binary protocols.

## Practice

Calculate how many complete boxes of 12 can be filled from 95 items and how many items remain. Predict the results before running.

# 6 · Decisions and Boolean Logic

## Problem

Convert a mark from 0 to 100 into a grade. First reject invalid marks. Then check thresholds from highest to lowest.

```
mark = 76
if not 0 <= mark <= 100:
    grade = "Invalid"
elif mark >= 80:
    grade = "A"
elif mark >= 70:
    grade = "B"
elif mark >= 60:
    grade = "C"
elif mark >= 50:
    grade = "D"
else:
    grade = "F"
print(grade)
```

**Expected output:** `B`

An `if`/`elif` chain takes only its first true branch. If you check `mark >= 50` first, a mark of 90 may receive the wrong grade. Correct logic depends on order.

## Flowchart: a decision

Start → check valid range? **No:** report invalid → End. **Yes:** check grade thresholds in descending order → display grade → End.

## Truth tables

| a | b | a and b | a or b |
|---|---|---|---|
| False | False | False | False |
| False | True | False | True |
| True | False | False | True |
| True | True | True | True |

This table uses Boolean operands. In general, Python's `and` and `or` return one of their operands, not necessarily `True` or `False`. `name or "Guest"` returns a non-empty name or the fallback string. `not` produces a Boolean.

## Short-circuit evaluation

Python stops when it already knows the result. In `denominator != 0 and numerator / denominator > 2`, the division is not evaluated when the denominator is zero. This is useful, but the order of the checks matters.

## Truthiness

Zero, `None` and empty containers are falsey. Most other values are truthy. `if items:` is often clearer than `if len(items) > 0:`. A whitespace-only string is non-empty; strip it if blank input should count as empty.

## Common mistakes

- Using `=` instead of `==` for comparison.
- Writing `if answer == "yes" or "y":`; the non-empty string `"y"` makes this condition truthy. Use `answer in {"yes", "y"}`.
- Testing only a middle value. Test boundaries such as 49, 50, 79, 80 and 100.

## Practice

Check whether a year is a leap year: divisible by 400, or divisible by 4 but not by 100. Test 1900, 2000 and 2024.

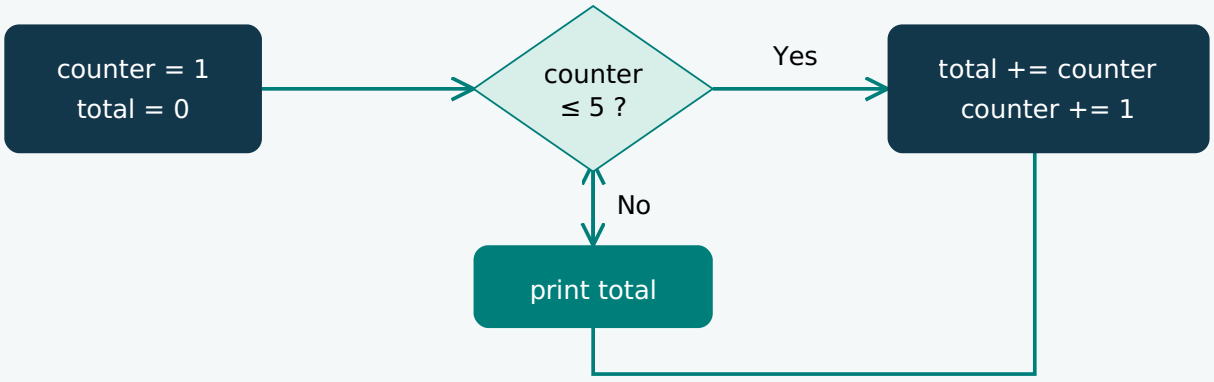
# 7 · Repetition with While

## Problem

Add the whole numbers from 1 through 5. We need a changing counter and a running total.

```
counter = 1
total = 0
while counter <= 5:
    total += counter
    counter += 1
print(total)
```

**Expected output:** 15




02 / A while loop checks its condition before every iteration.

## Trace table

| Before body: counter | Total before | Total after | Counter after |
|---|---|---|---|
| 1 | 0 | 1 | 2 |
| 2 | 1 | 3 | 3 |
| 3 | 3 | 6 | 4 |
| 4 | 6 | 10 | 5 |
| 5 | 10 | 15 | 6 |

The next condition is `6 <= 5`, which is false. The body does not run a sixth time.

## A loop needs progress

Ask three questions: What is true before the loop? What changes each iteration? Why will the condition eventually become false? If you omit `counter += 1`, the condition never changes and the program keeps running.

A **loop invariant** is a property that remains true at a consistent point. Here, before each iteration, `total` is the sum of the integers smaller than `counter` starting at 1. This is a useful way to reason about correctness without guessing.

## Sentinel loops

A sentinel is a special input meaning stop, such as `"quit"`. A menu can use `while True:` and `break` when the user chooses to exit. Ensure there is a reachable exit path. For invalid input, explain the problem and return to the prompt rather than silently repeating.

## Break, continue and pass

- `break` exits the nearest loop.
- `continue` skips the remainder of this iteration and starts the next.
- `pass` does nothing; it is a placeholder, not a request to skip an iteration.

## Practice

Trace a countdown from 3 to 1, then print `Go!`. Next, plan a password-attempt loop limited to three attempts. Do not use real passwords in a learning exercise.

# 8 · Iteration with For and Range

## Choose iteration when you already have items

A `for` loop obtains items from an iterable. It does not require you to manage an index manually. Strings, lists, dictionaries, sets and ranges are iterable.

```
scores = [65, 80, 75]
for position, score in enumerate(scores, start=1):
    print(f"Student {position}: {score}")
print("Even numbers:")
for number in range(2, 9, 2):
    print(number, end=" ")
print()
```

### Expected output

```
Student 1: 65
Student 2: 80
Student 3: 75
Even numbers:
2 4 6 8
```

`enumerate()` yields an index-counter and an item. `start=1` changes the counter, not the list's actual zero-based indexing.

## Range boundaries

`range(start, stop, step)` excludes `stop`. `range(5)` produces 0 through 4. `range(5, 0, -1)` produces 5 through 1. A zero step is invalid. A range is a compact iterable representation, not a prebuilt list of all the integers.

## Loop else

A loop's `else` runs when the loop finishes without `break`, even if it had zero iterations. It does not mean “the loop's condition was false on the first check.” It is useful for a search: break when found; otherwise report not found.

## Search plan

For each item → compare with target → if equal, report success and break. If the loop exhausts all items without a break → report absence. A function that returns immediately when found is often even clearer.

### Do not modify what you are traversing casually

Removing list items while iterating over the same list can skip elements. Build a new filtered list instead. Adding or removing dictionary entries during iteration can raise `RuntimeError`.

### Practice

Print the multiplication table of 7 from `7 × 1` to `7 × 10`. Then total all multiples of 3 below 20. Check whether your stop boundary includes 20 or excludes it.

# 9 · Tracing, Nested Loops and Patterns

## Why patterns matter

Patterns are not about memorizing pictures. They train you to connect a row number to the number of spaces, symbols and values that row needs. Every pattern should have a rule before it has code.

For a centered pyramid of height `n`, row `r` (starting at 1) needs `n - r` leading spaces and `2*r - 1` stars.

```
height = 4
for row in range(1, height + 1):
    spaces = height - row
    stars = 2 * row - 1
    print(" " * spaces + "*" * stars)
```

### Expected output

```
   *
  ***
 *****
*******
```

| Row | Leading spaces | Stars | Total printed width |
|---|---|---|---|
| 1 | 3 | 1 | 4 |
| 2 | 2 | 3 | 5 |
| 3 | 1 | 5 | 6 |
| 4 | 0 | 7 | 7 |

The visible pyramid is centered in an imagined width of 7. Trailing spaces are unnecessary, so the actual printed lengths differ.

## Nested-loop model: a hollow square

```
size = 4
for row in range(size):
    for column in range(size):
        border = row in (0, size - 1) or column in (0, size - 1)
        print("*" if border else " ", end="")
    print()
```

### Expected output

```
****
*  *
*  *
****
```

The outer loop selects a row. The inner loop visits every column in that row. The newline belongs after the inner loop. If it is inside the inner loop, every character gets its own line.

## A reusable pattern recipe

1. Choose coordinates: zero-based or one-based, not a mixture.
2. Describe the border or filled region using those coordinates.
3. Test the smallest meaningful size.
4. Trace one complete row.
5. Compare output in a monospaced font.

A right triangle has `r` stars on row `r`. An inverted right triangle has `n-r+1`. Floyd's triangle needs one counter that continues across rows. A diamond combines increasing odd widths with decreasing odd widths; do not repeat its widest row unless that is intentional.

## Complexity intuition

A square of side `n` visits `n*n` cells: time grows quadratically, written O(n²). A triangle also has quadratic growth even though it prints fewer cells. This notation describes growth, not an exact number of seconds.

## Practice

Draw and trace a height-3 diamond. Add a hollow pyramid only after you can explain its left edge, right edge and base condition. Solutions for several patterns appear in Chapter 30.

# 10 · Strings and Text Processing

## Text is a sequence

A string is immutable: operations produce new strings rather than changing its characters in place. Indexing starts at zero; negative indices count from the end.

```
text = "Python"
print(text[0], text[-1])
print(text[1:4])
print(text[::-1])
words = "  learn Python slowly  ".strip().split()
print(" | ".join(words))
print("banana".count("an"))
```

### Expected output

```
P n
yth
nohtyP
learn | Python | slowly
2
```

## Slice boundaries

`text[start:stop:step]` excludes the stop position. Omitting the start or stop uses a boundary appropriate to the direction. `[::-1]` traverses backward. An out-of-range index raises `IndexError`; a slice usually clips to the available bounds.

## Useful operations

| Operation | Purpose |
| --- | --- |
| `.strip()` | Remove surrounding whitespace |
| `.lower()` / `.upper()` | Change case |
| `.casefold()` | More thorough caseless comparison |
| `.replace(old, new)` | Return replaced text |
| `.split()` | Split on whitespace by default |
| `separator.join(parts)` | Combine strings with a separator |

| Operation | Purpose |
| --- | --- |
| `needle in text` | Test substring membership |
| `.startswith()` / `.endswith()` | Test boundaries |

`join()` requires strings; convert numbers explicitly. `split()` without an argument collapses runs of whitespace, while `split(" ")` uses literal spaces and may produce empty fields.

## Unicode awareness

Python strings store Unicode text, so Bengali and English can coexist. A Python character index is not always a visible letter or emoji boundary: combined marks can occupy multiple code points. Simple reversal is a sequence operation, not a universal text-layout algorithm.

## Formatting clearly

Prefer `f"{name}: {score}"` for straightforward interpolation. `.format()` is useful when a format template is stored separately. Do not mix automatic field numbering and explicit numeric field numbering in one `.format()` template.

## Practice

Normalize a user's answer with `.strip().casefold()`. Count words in a sentence using `.split()`. State that punctuation remains attached: a basic word count is not a full natural-language tokenizer.

# 11 · Lists and Mutable Objects

## Problem

Track scores that may change as corrections arrive. A list is an ordered, mutable sequence that allows duplicates.

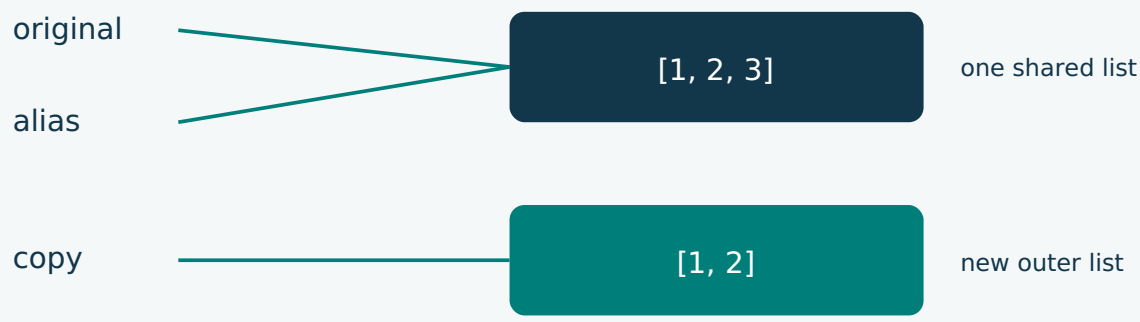
```
scores = [70, 85, 70]
scores.append(90)
scores[0] = 75
removed = scores.pop()
print(scores)
print(removed)
print(sorted(scores))
print(scores)
```

### Expected output

```
[75, 85, 70]
90
[70, 75, 85]
[75, 85, 70]
```

`append()` adds one object. `extend()` adds each item from an iterable. `pop()` removes and returns an item; `remove(value)` removes the first equal value and raises `ValueError` if absent. `sort()` changes the list in place and returns `None`; `sorted()` returns a new list.

## References and copying




03 / Assignment shares a reference; a shallow copy creates a distinct outer list.

```
original = [1, 2]
alias = original
copy = original.copy()
alias.append(3)
print(original)
print(copy)
```

**Expected output**

```
[1, 2, 3]
[1, 2]
```

Both `alias` and `original` refer to one list. `copy()` makes a new outer list, but it is a **shallow** copy: nested mutable objects are still shared. Avoid `[[0] * 3] * 3` for an independently editable grid. Use `[[0] * 3 for _ in range(3)]` so each row is a distinct list.

## Safe statistics

`sum(values) / len(values)` needs a non-empty list. `max()` and `min()` also need data unless you supply an appropriate default. Decide whether an empty dataset is an error, a special result or a request for more input.

“Second largest” is ambiguous. Does `[9, 9, 7]` have second largest 9 by position or 7 among distinct values? State the interpretation. A distinct-value version needs at least two unique values.

## Practice

Given five scores, produce a new list containing only scores at least 50. Keep the original unchanged. Then explain why `result = scores.sort()` does not store a sorted list in `result`.

# 12 · Tuples and Unpacking

## Fixed group, not magical protection

A tuple is an immutable sequence. It is useful for returning multiple related results or representing a coordinate. The comma creates the tuple; a one-item tuple is `(5,)`, not `(5)`.

```
point = (3, 4)
x, y = point
print(x + y)
first, *middle, last = [10, 20, 30, 40]
print(first, middle, last)
a, b = 1, 2
a, b = b, a
print(a, b)
```

### Expected output

```
7
10 [20, 30] 40
2 1
```

Python evaluates the right-hand side of the swap before rebinding the names on the left. Unpacking requires matching lengths unless you use one starred target to collect the remaining items.

## Immutability has a boundary

You cannot replace an element of a tuple. However, if a tuple contains a list, that contained list can still change. Similarly, a tuple is hashable only when all of its elements are hashable. `(1, 2)` can be a dictionary key; `(1, [2])` cannot.

Choose a tuple to communicate a fixed grouping, not because of a blanket claim that it is always faster. For named fields, a dataclass may be clearer than remembering that index 0 means a name and index 1 means a score.

## Practice

Return the quotient and remainder of dividing 17 by 5, then unpack them into meaningful names. Try Python's built-in `divmod()` after writing the two operations yourself.

# 13 · Dictionaries and Records

## Problem

Look up a learner's score by name. A dictionary maps unique, hashable keys to values. In modern Python, dictionaries preserve insertion order, but that does not mean they automatically sort keys.

```
scores = {"Amina": 82, "Rafi": 75}
scores["Nila"] = 91
scores["Rafi"] = 78
print(scores["Rafi"])
print(scores.get("Sami", "Not recorded"))
for name, score in scores.items():
    print(f"{name}: {score}")
```

### Expected output

```
78
Not recorded
Amina: 82
Rafi: 78
Nila: 91
```

`dictionary[key]` raises `KeyError` when the key is absent. `.get(key, default)` handles that specific missing-key situation; it does not prevent every possible exception.

## Counting as accumulation

Start with an empty dictionary. For each word, obtain its previous count or zero, add one, and save it. This is the same accumulation idea used for totals, now with one total per key.

```
counts = {}
for word in "red blue red green blue red".split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

**Expected output:** `{'red': 3, 'blue': 2, 'green': 1}`

## Records and collections of records

A student record might be `{"name": "Amina", "score": 82}`. Many students can be a list of such dictionaries. Use consistent keys and validate values at the input boundary. Nesting is useful, but too many levels can make code harder to understand; introduce functions or classes when the structure grows.

## Merge and update

In Python 3.9+, `left | right` makes a merged dictionary. When both contain a key, the right-hand value wins. `left |= right` updates `left`. Neither operation recursively merges nested dictionaries.

## Practice

Build a contact dictionary with fictional names and phone-number strings. Add one contact, update another and look up a missing name safely. Do not use real private contact data in sample files.

# 14 · Sets and Membership

## Use a set for uniqueness

A set is a mutable collection of unique, hashable elements. It does not provide positional indexing or a guaranteed display order. `set()` creates an empty set; `{}` creates an empty dictionary.

```
python_students = {"Amina", "Rafi", "Nila"}
web_students = {"Rafi", "Sami"}
print(sorted(python_students & web_students))
print(sorted(python_students | web_students))
print(sorted(python_students - web_students))
python_students.add("Sami")
python_students.discard("Unknown")
print(len(python_students))
```

### Expected output

```
['Rafi']
['Amina', 'Nila', 'Rafi', 'Sami']
['Amina', 'Nila']
4
```

Sorting here makes the displayed examples deterministic. It does not change the set into an ordered collection. Union `|` combines members, intersection `&` keeps shared members, difference `-` keeps members only on the left, and symmetric difference `^` keeps members appearing in exactly one set.

## Remove or discard?

`remove(value)` raises `KeyError` for an absent member. `discard(value)` does nothing when the member is absent. Choose based on whether absence represents a bug or an acceptable situation.

## Hashability

Numbers, strings and suitable tuples can be set members. Lists, dictionaries and ordinary sets cannot. A `frozenset` is an immutable set and can be hashable. Hashability is a contract about stable hash/equality behavior; it is more specific than the casual phrase “cannot be changed.”

## Preserve order when needed

`list(set(items))` removes duplicates but does not promise to retain first-seen order. For hashable values in modern Python, `list(dict.fromkeys(items))` preserves that order. A loop with a `seen` set also makes the intention explicit.

## Practice

Compare the subjects chosen by two fictional students. Print shared subjects and subjects chosen by only one. Sort output so tests do not depend on set iteration order.

# 15 · Functions, Arguments and Return Values

## Problem

Several parts of a program need to calculate a discounted price. Repeating the formula invites inconsistent fixes. Put the rule in one function.

```
def discounted_price(price, percent=10):
    """Return price after a percentage discount."""
    if price < 0:
        raise ValueError("price cannot be negative")
    if not 0 <= percent <= 100:
        raise ValueError("percent must be between 0 and 100")
    return price * (1 - percent / 100)

print(f"{discounted_price(200):.2f}")
print(f"{discounted_price(200, percent=25):.2f}")
```

### Expected output

```
180.00
150.00
```

A **parameter** is a name in a function definition. An **argument** is a value supplied in a call. A default makes an argument optional. A keyword argument names the parameter explicitly and often improves readability.

## Return is not print

`return` hands a value to the caller and ends that function call. `print` sends a representation to output. A function that only prints is harder to reuse in a calculation, GUI or test. A function without an explicit return value returns `None`.

## Good function boundaries

Prefer one clear responsibility. A `calculate_total()` function should usually calculate, not also prompt the user and write a file. Separate pure logic from input/output. This makes logic easier to test.

## Defaults and mutation

Default values are evaluated once when the function is defined, not each time it is called. Do not use a mutable default such as `items=[]` when you expect a fresh list per call. Use `items=None`, then create a new list inside the function when needed.

## Flexible arguments

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. These names are conventions, not keywords. They are useful for wrappers and flexible interfaces, but explicit parameters are easier for beginners and tools to understand.

## Multiple results

`return minimum, maximum` returns one tuple, which the caller can unpack. Choose a dictionary or dataclass if many named results would be clearer.

## Practice

Write `is_even(number)` returning a Boolean. Then use it in both a conditional and a list-filtering loop. The function itself should not call `input()` or `print()`.

# 16 · Scope, Recursion and Problem Decomposition

## Local names

Names assigned inside a function are normally local to that call. Python looks for names through local, enclosing-function, global-module and built-in scopes, often remembered as LEGB.

`global` refers to a module-level binding, not any arbitrary outer variable. `nonlocal` refers to an existing binding in an enclosing function scope. Prefer passing values and returning results when that makes dependencies clear.

## Problem: factorial

For a non-negative integer `n`, factorial is `1 × 2 × ... × n`, with `0! = 1`. A recursive definition is `factorial(n) = n * factorial(n - 1)` with a base case at 0.

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

**Expected output:** `24`

## Trace the calls

`factorial(4)` waits for `factorial(3)`, which waits for 2, then 1, then 0. The base case returns 1. Results unwind as 1, 2, 6 and 24. Each call has its own local `n`.

The function assumes an integer argument. Chapter 20 tests contracts; Chapter 24 explains that type hints alone do not enforce them. A public-facing function may need explicit type validation as well as range validation.

## When recursion helps—and when it does not

Recursive structure fits tree traversal and nested data. For factorial, a loop is usually simpler and avoids Python's recursion-depth limit. Do not increase that limit just to force a beginner calculation to work. Naive recursive Fibonacci repeats work; an iterative version or memoization is more suitable for larger inputs.

## Decompose a bigger task

An expense tracker can become: parse an amount → validate a record → calculate totals → load records → save records → handle menu choices. These are meaningful functions, not arbitrary chunks of ten lines.

## Practice

Implement factorial with a loop. Test 0, 1, 5 and a negative value. Explain how the loop's initial product of 1 corresponds to the recursive base case.

# 17 · Modules, Packages and Virtual Environments

## Reuse without copying

A module is a Python file that can be imported. A package organizes modules. The standard library ships with Python and includes tools such as `math`, `random`, `json`, `csv`, `pathlib` and `unittest`.

```
from math import isqrt
import random

print(isqrt(81))
generator = random.Random(7)
choices = ["rock", "paper", "scissors"]
choice = generator.choice(choices)
print(choice in choices)
```

### Expected output

```
9
True
```

Use `random` for games and simulations, not passwords or security tokens. Use `secrets` for security-sensitive randomness. Seeding is useful for reproducible learning examples but does not make randomness secure.

## The main guard

When a file runs as the entry point, its `__name__` is `"__main__"`. When imported, `__name__` is normally its module name. Put interactive startup behind `if __name__ == "__main__":` so importing helper functions does not unexpectedly launch a menu.

## Isolate dependencies

A virtual environment isolates project package installations. The built-in `venv` module is sufficient for this book; installing `virtualenv` is not required.

```
python -m venv .venv
```

Activate it using the command for your shell:

| Platform/shell | Command |
| --- | --- |
| Windows PowerShell | `.venv\Scripts\Activate.ps1` |
| Windows Command Prompt | `.venv\Scripts\activate.bat` |
| macOS/Linux bash or zsh | `source .venv/bin/activate` |

If PowerShell blocks activation, do not disable security settings indiscriminately. You can invoke `.venv\Scripts\python.exe` directly without activation. On Unix-like systems the equivalent is `.venv/bin/python`.

```
python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

Use `deactivate` to leave an activated environment. `pip freeze` records installed distributions and versions; it is not a complete cross-platform lockfile or a guarantee that every environment will be identical. The projects supplied here use the standard library and do not require `requests`.

## Common mistakes

- Naming your file `random.py` or `json.py`, shadowing the standard module.
- Installing into one interpreter and running another.
- Sharing `.venv` as source code. Recreate it instead.
- Committing credentials in a project folder.

## Practice

Create a virtual environment in a fresh folder. Confirm its interpreter, then create a small module containing a function and import it from a second file.

# 18 · Files, Paths, CSV and JSON

## Data should survive the program

Variables disappear when the process ends. A file lets a later run recover information. Use `pathlib.Path` for readable path handling and a `with` statement to close a file even when ordinary exceptions occur.

```
from pathlib import Path
import json

path = Path("learning-demo.json")
record = {"name": "Amina", "topics": ["loops", "functions"]}
with path.open("w", encoding="utf-8") as file:
    json.dump(record, file, ensure_ascii=False, indent=2)
with path.open("r", encoding="utf-8") as file:
    loaded = json.load(file)
print(loaded["name"])
print(len(loaded["topics"]))
```

### Expected output

```
Amina
2
```

This example creates or replaces `learning-demo.json` in the current working directory. Run examples in a practice folder, not among irreplaceable files.

## File modes

- `"r"`: read; fail if absent.
- `"w"`: write; truncate an existing file.
- `"a"`: append; create if absent.
- `"x"`: create exclusively; fail if already present.
- Add `"b"` for binary data, such as `"rb"`.

Relative paths are resolved from the current working directory, which may differ from the script's folder. For a resource beside a script, start with `Path(__file__).resolve().parent`.

## Text, CSV and JSON

Plain text is good for notes. CSV is useful for rows and columns; use `csv.reader` or `csv.DictReader` instead of splitting on commas manually because quoted fields may contain commas. When opening CSV files, use `newline=""` and an explicit encoding.

JSON stores objects, arrays, strings, numbers, Booleans and null. Python dictionaries usually become JSON objects, and lists become arrays. JSON object keys are strings. A Python set is not directly JSON serializable. Tuples round-trip as lists unless you add your own conversion rules.

## Loading is not validation

A valid JSON file can still contain the wrong structure. Check whether you received a list of records, whether required keys exist and whether amounts have valid values. Do not use `eval()` to read data. It can execute code.

## Safer saving

Writing directly to an important file can leave partial contents if interrupted. A common improvement is writing a temporary file in the same directory, then replacing the destination. This reduces partial-file risk, but it is not a substitute for backups, transaction handling or multi-process coordination.

## Practice

Write three study goals to a UTF-8 text file, one per line. Read them back and number them. Test the missing-file case after Chapter 19.

# 19 · Exceptions, Validation and Debugging

## Expect imperfect input

Validation checks whether input meets a rule. Exception handling recovers from operations that could not complete. They work together; neither is a replacement for the other.

```
def parse_positive_integer(text):
    try:
        value = int(text)
    except ValueError as error:
        raise ValueError("enter a whole number") from error
    if value <= 0:
        raise ValueError("number must be positive")
    return value

for text in ["12", "0", "twelve"]:
    try:
        print(parse_positive_integer(text))
    except ValueError as error:
        print(f"Invalid: {error}")
```

### Expected output

```
12
Invalid: number must be positive
Invalid: enter a whole number
```

## Exception structure

`try` encloses the operation that may fail. `except` handles specified exception types. `else` runs if the try block completes normally without an exception. `finally` runs as control leaves the try statement in normal Python execution, including most error and return paths. Abrupt process termination, a power failure or `os._exit()` is outside that ordinary guarantee.

Catch specific exceptions near the operation you understand. A bare `except:` also catches exit-related exceptions such as `KeyboardInterrupt`; using it everywhere can make a program difficult to stop. Even `except Exception:` is too broad when it hides a programming bug.

## Read a traceback from the bottom

The last line identifies the exception type and message. The stack frames above it show where calls led. Find the relevant line in your code, inspect the values involved, and reproduce the smallest case that fails.

| Symptom | Likely question |
| --- | --- |
| `NameError` | Was the name defined in this scope? |
| `TypeError` | Do these values support this operation? |
| `IndexError` | Is this index inside the sequence? |
| `KeyError` | Does this key exist? |
| `ValueError` | Is the value acceptable to this operation? |
| Wrong result, no exception | Is the algorithm or boundary wrong? |

## Debugging procedure

1. State the expected result exactly.
2. Make a minimal failing example.
3. Inspect values with a temporary print or `breakpoint()`.
4. Trace one iteration or function call.
5. Fix the cause, not just the symptom.
6. Add a test so the bug does not quietly return.

Division by zero is undefined in ordinary arithmetic. Do not present it as a mathematically valid infinity merely because a practice question requests a message saying “infinite.” Report that the operation is not defined.

## Practice

Handle a missing text file with `FileNotFoundError` and report the path. Do not catch all exceptions and pretend an empty file was loaded successfully.

# 20 · Testing and Program Design

## Test rules, not just happy paths

A useful test has an input, an expected result and a reason. Consider normal values, boundaries, empty data, duplicates and invalid values. Small pure functions are easy to test because they do not depend on keyboard timing, networks or hidden global state.

```
import unittest


def second_largest_distinct(values):
    unique = sorted(set(values))
    if len(unique) < 2:
        raise ValueError("need at least two distinct values")
    return unique[-2]


class RankingTests(unittest.TestCase):
    def test_normal(self):
        self.assertEqual(second_largest_distinct([4, 9, 7]), 7)

    def test_duplicates(self):
        self.assertEqual(second_largest_distinct([9, 9, 7]), 7)

    def test_insufficient(self):
        with self.assertRaises(ValueError):
            second_largest_distinct([5, 5])


if __name__ == "__main__":
    unittest.main()
```

Run this file with Python. It reports three tests and `OK` when they pass. Unittest progress is normally written to the standard error stream, which is still visible in a terminal.

## What the tests establish

They check a specific interpretation: the second largest **distinct** value. They do not establish behavior for mixed incomparable types, huge datasets or arbitrary unhashable objects. A function contract should state its expected domain. Here it is a finite collection of comparable, hashable numbers.

## Assertions and production validation

`assert condition` is convenient during development, but Python can remove assertions in optimized mode. Do not rely on them to reject untrusted input. Use explicit checks and exceptions for required validation.

## Design for change

Keep parsing, calculation and presentation separate. Give functions meaningful names. Avoid a single giant loop containing every feature. A few clear helpers are better than a complicated hierarchy introduced before it solves a real problem.

## Complexity basics

A single pass is often O(n). A sort is typically O(n log n). A double nested scan may be $O(n^2)$. Dictionary and set membership are average O(1) under ordinary hashing assumptions, not an unconditional guarantee. Optimize only after the result is correct and the bottleneck is measured.

## Practice

Write tests for an average function. Decide explicitly what an empty list should do. Add a test for that decision before writing the implementation.

# 21 · Classes and Object-Oriented Thinking

## Bundle related state and behavior

A class defines a kind of object. An instance is one particular object of that kind. Use a class when state and the operations that protect or change it belong together—not merely because every noun could become a class.

```
class StudySession:
    category = "learning"

    def __init__(self, topic):
        self.topic = topic
        self.minutes = 0

    def add_minutes(self, amount):
        if amount <= 0:
            raise ValueError("minutes must be positive")
        self.minutes += amount

    def summary(self):
        return f"{self.topic}: {self.minutes} minutes"


session = StudySession("Python")
session.add_minutes(25)
session.add_minutes(10)
print(session.summary())
```

**Expected output:** `Python: 35 minutes`

## Follow an instance method call

`session.add_minutes(25)` calls the function defined in the class and supplies the instance as its first argument, conventionally named `self`. `self` is not a keyword, but changing the convention makes code harder to read.

`__init__` initializes an instance after creation. It is often called a constructor informally; strictly, object creation is handled by `__new__`. `__init__` must return `None`, not the newly created object.

## Class and instance attributes

`category` is shared through the class. `topic` and `minutes` are stored on each instance. Assigning `session.category = "practice"` normally creates an instance attribute that shadows the class attribute; it does not change the class value for all instances.

Avoid a class-level list when every instance should have its own independent list. Create that list in `__init__` instead.

## Three method styles

- An instance method receives `self` and works with instance state.
- A `@classmethod` receives `cls` and can build alternative constructors.
- A `@staticmethod` receives neither automatically; it is a utility grouped with the class.

## Practice

Create a `Book` class with title and author. Add a method returning a readable description. Then make two instances and confirm that changing one title does not change the other.

# 22 · Inheritance, Properties and Special Methods

## Inheritance models a substitutable kind

A child class can extend a parent class, but “is a” should mean more than a convenient reuse shortcut. If a derived object cannot behave sensibly where a base object is expected, composition may be clearer. Composition means an object contains another object and delegates work to it.

```
class Resource:
    def __init__(self, title):
        self.title = title

    def describe(self):
        return self.title


class Video(Resource):
    def __init__(self, title, minutes):
        super().__init__(title)
        self.minutes = minutes

    def describe(self):
        return f"{self.title} ({self.minutes} min)"


lesson = Video("Loops", 12)
print(lesson.describe())
print(isinstance(lesson, Resource))
```

### Expected output

```
Loops (12 min)
True
```

`super()` follows the method resolution order (MRO). In a simple single-inheritance example it reaches the parent method. In multiple inheritance it is not simply “call a fixed parent”; cooperative methods must respect the MRO and compatible calling conventions.

Single inheritance has one direct base; multiple inheritance has several; multilevel inheritance forms a chain. Beginners should prefer small, shallow structures until a concrete need justifies more.

### Properties protect an invariant

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

    def __str__(self):
        return f"{self.celsius:.1f} C"


print(Temperature(25))
```

**Expected output:** `25.0 C`

The setter validates before storing. `_celsius` signals an internal detail by convention; it is not a security boundary. Do not create getter and setter methods mechanically for every attribute. Plain public attributes are often appropriate.

### Special methods

`__str__` supplies a readable string. `__repr__` supplies a useful developer representation. `__len__` must return a non-negative integer. `__add__` can implement `+`; return `NotImplemented` for unsupported operand types so Python can attempt the appropriate reflected operation. Do not invent surprising operator meanings just because you can.

### Dataclasses

For a simple record, `@dataclass` can generate initialization and representation methods. It reduces boilerplate but does not automatically validate types or turn a mutable design into a safely immutable one.

### Practice

Design a small vector with x and y coordinates. Plan vector addition and string representation. Decide what adding a vector and a string should do before implementing `__add__`.

# 23 · Comprehensions, Iterators and Functional Tools

## Transform, select, accumulate

These are three common jobs: transform every item, select items meeting a condition, or combine items into one result. Prefer the clearest expression of the job.

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
squares = [number * number for number in numbers]
even = [number for number in numbers if number % 2 == 0]
print(squares)
print(even)
print(list(map(lambda number: number * 2, numbers)))
print(list(filter(lambda number: number > 3, numbers)))
print(reduce(lambda total, number: total + number, numbers, 0))
print(sum(number * number for number in numbers))
```

### Expected output

```
[1, 4, 9, 16, 25]
[2, 4]
[2, 4, 6, 8, 10]
[4, 5]
15
55
```

## Comprehension structure

Read `[expression for item in iterable if condition]` as: for each item, keep it if the condition passes, then calculate the expression. The loop variable must match the name used in the expression. `[i for item in values]` is wrong unless a deliberately separate `i` exists—and then it probably means something unintended.

Set and dictionary comprehensions follow similar ideas. Do not compress several nested loops and conditions into one unreadable line. A normal loop is not a failure.

## Iterators and generators

`map()` and `filter()` return iterators in Python 3, not lists. They are consumed as you iterate. A generator expression such as `(n*n for n in numbers)` also produces values lazily. A generator function uses `yield` to pause and resume its state.

Laziness can reduce memory use, but it can also postpone an exception until iteration. If you need to traverse results repeatedly, materialize a list or create a new iterator.

## Lambda, closure and decorators

A lambda is a small single-expression function. Use `def` when logic needs a name, documentation or several statements. A closure retains access to an enclosing function's variables. A decorator takes a callable and returns a replacement callable; it can add logging or timing without changing every call site. Decorators are an extension topic, not a prerequisite for your first useful program.

`reduce()` combines a running result with each next item. For addition, prefer `sum()`; for a maximum, prefer `max()`. Use `reduce()` to understand accumulation, not to make simple operations obscure.

## Practice

Turn a list of names into stripped, non-empty names using a readable loop first. Then write a comprehension. Explain why filtering before stripping may retain whitespace-only entries.

# 24 · Type Hints, Match and Assignment Expressions

## Hints describe intent

Type annotations help readers and static analysis tools understand expected values. Python does not automatically enforce them at runtime.

```
def describe_status(code: int) -> str:
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not found"
        case 500:
            return "Server error"
        case _:
            return "Unknown status"


scores: list[int] = [72, 81, 90]
identifier: int | str = "ID-101"
print(describe_status(404))
if (count := len(scores)) >= 3:
    print(f"Recorded {count} scores")
```

### Expected output

```
Not found
Recorded 3 scores
```

Built-in generic annotations such as `list[int]` are available in Python 3.9+. `int | str` and `match` require Python 3.10+. Older references may use `typing.List` or `typing.Union`; recognize them, but use the modern forms for the stated baseline.

## Match is structural pattern matching

The simple literal cases above resemble a switch, but `match` can also match sequences, mappings and object structures. A bare name in a pattern usually captures a value rather than comparing with an existing variable. Use literal patterns or qualified constants when comparison is intended. No `break` is required to avoid falling through to the next case.

## The walrus operator

`:=` assigns a value inside an expression. It arrived in Python 3.8. In the example, it lets us compute a count and then use it in both the test and the message. Do not use it just to shorten a line. Ordinary assignment is often clearer.

## Runtime contracts still matter

An annotation `amount: int` does not reject `"ten"`. Use explicit validation at input boundaries. A static type checker can flag many mistakes before execution, but passing a type check is not proof that an algorithm is correct.

## Practice

Annotate an average function accepting `list[float]` and returning `float`. Define its empty-input behavior. Then explain which part is checked by your runtime code and which part is only a hint.

# 25 · Project: A Resilient Calculator

## Goal and contract

Accept two finite numbers and one operation. Calculate only the operations the program explicitly supports. Recover from invalid input and exit cleanly. Do not execute input as code with `eval()`.

**Flow:** Start → read first value or quit → read operation → read second value → validate and calculate → display result or helpful error → return to prompt.

```
import math


def calculate(left, operator, right):
    if not math.isfinite(left) or not math.isfinite(right):
        raise ValueError("numbers must be finite")
    if operator == "+":
        result = left + right
    elif operator == "-":
        result = left - right
    elif operator == "*":
        result = left * right
    elif operator == "/":
        if right == 0:
            raise ValueError("division by zero is undefined")
        result = left / right
    else:
        raise ValueError("choose +, -, * or /")
    if not math.isfinite(result):
        raise ValueError("result is too large")
    return result


def main():
    print("Calculator — type q as the first value to quit.")
    while True:
        first = input("First value: ").strip()
        if first.casefold() == "q":
            print("Goodbye.")
            break
        try:
            left = float(first)
            operator = input("Operation (+ - * /): ").strip()
            right = float(input("Second value: "))
            result = calculate(left, operator, right)
        except ValueError as error:
            print(f"Cannot calculate: {error}")
        else:
            print(f"Result: {result:g}")
```

```
if __name__ == "__main__":
    main()
```

## Example session

Entering `12`, `/`, `4` displays `Result: 3`. Entering `12`, `/`, `0` displays a message explaining that division by zero is undefined. Entering `q` at the first prompt exits.

## Why the design works

`calculate()` contains the numerical rules and can be tested without a keyboard. `main()` handles the conversation. Catching `ValueError` covers expected parsing and domain errors without hiding unrelated programmer mistakes. Finite checks reject `nan` and infinity, which `float()` can otherwise parse.

This calculator uses floats and is not a financial accounting system. Rounding the display does not turn approximate arithmetic into exact decimal arithmetic.

## Test plan

| Case | Expected behavior |
|---|---|
| `12 / 4` | 3 |
| `2 - 5` | -3 |
| Division by zero | Helpful error |
| Unsupported operator | Helpful error |
| Non-numeric input | Helpful error, menu continues |
| `nan` or infinity | Rejected |
| Quit | Loop ends |

## Extend it

Add exponentiation only after defining how negative bases, fractional exponents and overflow should behave. Add history by returning records from the input layer, not by making the calculation function secretly write files.

# 26 · Project: A Quiz with a Score Report

## Goal

Ask several questions, normalize answers, count correct responses and show missed items. Store questions as data so the quiz engine does not require a new branch for every question.

```
QUESTIONS = [
    {"prompt": "Type of input() result? ", "answer": "str"},
    {"prompt": "Result of 7 // 2? ", "answer": "3"},
    {"prompt": "Keyword that returns a function result? ", "answer": "return"},
]


def normalize(text):
    return text.strip().casefold()


def run_quiz(questions):
    correct = 0
    missed = []
    for number, question in enumerate(questions, start=1):
        response = input(f"{number}. {question['prompt']}")
        if normalize(response) == normalize(question["answer"]):
            correct += 1
            print("Correct.")
        else:
            print("Not this time.")
            missed.append(question)
    return correct, missed


def main():
    if not QUESTIONS:
        print("No questions available.")
        return
    correct, missed = run_quiz(QUESTIONS)
    percentage = correct / len(QUESTIONS) * 100
    print(f"Score: {correct}/{len(QUESTIONS)} ({percentage:.1f}%)")
    for question in missed:
        print(f"Review: {question['prompt']}Answer: {question['answer']}")


if __name__ == "__main__":
    main()
```

### Trace a mixed attempt

Inputs `str`, `3` and `print` earn 2 out of 3. The spaces around `str` disappear during normalization. The final response is incorrect because displaying a value is not the same as returning it. The report displays `66.7%` after rounding for presentation.

### Design lessons

The questions are data. The loop is the algorithm. Keeping them separate allows a future version to load a JSON question bank without rewriting the score logic. The empty-data check prevents division by zero and gives the user a useful message.

Normalization is a deliberate policy. It is appropriate for keyword answers, but not for every subject: a case-sensitive identifier or exact output question might require stricter comparison. Do not erase distinctions that the question is meant to test.

### Test plan and extensions

Try all-correct, all-wrong and mixed attempts. Test capitalization, surrounding spaces and an empty question list. Add accepted alternatives as a list only when they are genuinely equivalent. Add shuffling with a local random generator; ensure the answers stay attached to their questions.

# 27 · Project: An Expense Tracker with JSON Persistence

## Goal

Create a local command-line tracker that adds expenses, lists them, summarizes totals by category and saves them between runs. It uses integer **paisa** as minor units: 100 paisa = 1 taka. It is an educational single-user tool, not a banking application.

## Plan before coding

A record has three fields: ISO date text, category text and a positive integer amount in minor units. The menu handles conversation. Parsing converts the amount. Loading checks saved data rather than trusting it. Saving writes a temporary sibling file and replaces the destination.

**Flow:** load → show menu → add/list/summary/quit → validate requested action → save after additions → return to menu.

```
from datetime import date
from decimal import Decimal, InvalidOperation, localcontext
import json
from pathlib import Path

DATA_PATH = Path(__file__).resolve().with_name("expenses.json")


def parse_amount(text):
    if len(text) > 64:
        raise ValueError("amount text is too long")
    try:
        amount = Decimal(text)
    except InvalidOperation as error:
        raise ValueError("enter a decimal amount") from error
    if not amount.is_finite() or amount <= 0:
        raise ValueError("amount must be positive and finite")
    if amount > Decimal("1000000000"):
        raise ValueError("amount exceeds this demo's limit")
    if amount < Decimal("0.01"):
        raise ValueError("minimum amount is one paisa")
    with localcontext() as context:
        context.prec = max(28, len(amount.as_tuple().digits) + 3)
        minor = amount * 100
        if minor != minor.to_integral_value():
            raise ValueError("use at most two decimal places")
        return int(minor)


def money(minor):
    return f"Tk {minor // 100}.{minor % 100:02d}"
```

```
def validate_records(records):
    if not isinstance(records, list):
        raise ValueError("saved data must be a list")
    for record in records:
        if not isinstance(record, dict):
            raise ValueError("each record must be an object")
        if set(record) != {"date", "category", "amount_minor"}:
            raise ValueError("record fields do not match")
        if not isinstance(record["date"], str):
            raise ValueError("date must be text")
        try:
            parsed = date.fromisoformat(record["date"])
        except ValueError as error:
            raise ValueError("invalid saved date") from error
        if parsed.isoformat() != record["date"]:
            raise ValueError("use YYYY-MM-DD dates")
        category = record["category"]
        if not isinstance(category, str) or not category.strip():
            raise ValueError("category must be non-empty text")
        amount = record["amount_minor"]
        if type(amount) is not int or not 0 < amount <= 100000000000:
            raise ValueError("invalid minor-unit amount")
    return records


def load_records(path):
    try:
        with path.open("r", encoding="utf-8") as file:
            records = json.load(file)
    except FileNotFoundError:
        return []
    return validate_records(records)


def save_records(path, records):
    validate_records(records)
    temporary = path.with_suffix(path.suffix + ".tmp")
    with temporary.open("w", encoding="utf-8") as file:
        json.dump(records, file, ensure_ascii=False, indent=2)
        file.write("\n")
    temporary.replace(path)


def totals_by_category(records):
    totals = {}
    for record in records:
        category = record["category"]
        totals[category] = totals.get(category, 0) + record["amount_minor"]
    return totals


def main():
    try:
        records = load_records(DATA_PATH)
    except (OSError, ValueError) as error:
        print(f"Cannot load data: {error}")
```

```
            print("Repair or back up the file first. Nothing was overwritten.")
            return
        while True:
            print("\n1 Add | 2 List | 3 Summary | 4 Quit")
            choice = input("Choice: ").strip()
            if choice == "1":
                category = input("Category: ").strip().casefold()
                try:
                    if not category:
                        raise ValueError("category cannot be blank")
                    amount = parse_amount(input("Amount in taka: ").strip())
                    new_record = {
                        "date": date.today().isoformat(),
                        "category": category,
                        "amount_minor": amount,
                    }
                    updated = records + [new_record]
                    save_records(DATA_PATH, updated)
                except (OSError, ValueError) as error:
                    print(f"Not saved: {error}")
                else:
                    records = updated
                    print("Saved.")
            elif choice == "2":
                if not records:
                    print("No expenses yet.")
                for record in records:
                    print(record["date"], record["category"],
                          money(record["amount_minor"]))
            elif choice == "3":
                totals = totals_by_category(records)
                for category, amount in sorted(totals.items()):
                    print(f"{category}: {money(amount)}")
                print(f"Total: {money(sum(totals.values()))}")
            elif choice == "4":
                print("Goodbye.")
                break
            else:
                print("Choose 1, 2, 3 or 4.")


    if __name__ == "__main__":
        main()
```

## Understand the important decisions

**Exact amounts:** `Decimal` parses decimal text without first converting through a float. The result is converted to integer minor units. `12.50` becomes 1250; `0.001` is rejected instead of silently rounded. The demonstration bounds input length and imposes a one-billion-taka per-record limit. A local decimal context preserves every supplied significant digit during conversion, so a long fractional value cannot silently round into a valid amount.

**No accidental data loss:** a malformed file does not become an empty dataset. The program reports the problem and stops. Otherwise, a later save could erase information the program failed to understand.

**Save before committing in memory:** `updated` is a candidate list. The active `records` reference changes only after saving succeeds. This keeps a failed addition from appearing as though it was safely persisted.

**Date handling:** entries use the computer's local date. The loader requires canonical `YYYY-MM-DD`. The application does not implement timezone-aware accounting or backdated entries.

**Boolean trap:** `bool` is a subclass of `int`, so `isinstance(True, int)` is true. The validator uses `type(amount) is int` to reject JSON Boolean amounts.

## Example session

Add category `food` and amount `120.50`, then category `travel` and amount `30`. Summary shows `food: Tk 120.50`, `travel: Tk 30.00` and `Total: Tk 150.50`. Quit and restart: the records should still be present beside the script.

## Limitations and next improvements

Only run one instance against a data file. The fixed temporary filename and replacement approach do not implement multi-user locking or full crash durability. Keep backups. This demo has no authentication, encryption, edit/delete interface or database transactions. A serious version should consider SQLite, backups, migrations and privacy requirements.

## Test plan

Test empty storage, a valid save/reload cycle, two categories, fractional-paisa input, negative input, infinity, malformed JSON, invalid record fields and a path that cannot be written. Use a temporary directory for tests so experiments never touch real expenses.

# 28 · Project: A Safe Assistant and Reply-Draft Workflow

### Start small and keep the human in control

A voice assistant and an auto-reply chatbot combine many independent systems: microphone access, recognition, text-to-speech, network services, credentials, chat-history parsing and user-interface automation. Build a reliable text core first. Add each external component behind an interface only after the core works.

This project intentionally creates **reply drafts** rather than sending messages. It uses only the standard library and works offline. It is a foundation for assistant logic, not a claim to implement a complete voice-recognition or language-model system.

```
from datetime import date


def draft_reply(message):
    normalized = message.strip().casefold()
    if not normalized:
        raise ValueError("message cannot be empty")
    if "thank" in normalized:
        return "You're welcome! Happy to help."
    if "meeting" in normalized:
        return "Thanks for the update. Please confirm the time and agenda."
    if normalized in {"hi", "hello", "hey"}:
        return "Hello! How can I help?"
    return "Thanks for your message. I'll review it and reply soon."


def respond(command):
    normalized = command.strip().casefold()
    if normalized == "help":
        return "Commands: date, draft <message>, help, quit"
    if normalized == "date":
        return date.today().isoformat()
    if normalized.startswith("draft "):
        message = command.strip()[6:]
        return "DRAFT ONLY: " + draft_reply(message)
    return "Unknown command. Type help."


def main():
    print("Local assistant. No message is sent automatically.")
    while True:
        command = input("> ")
        if command.strip().casefold() == "quit":
            print("Goodbye.")
            break
        try:
```

```
            print(respond(command))
        except ValueError as error:
            print(f"Cannot draft: {error}")


if __name__ == "__main__":
    main()
```

## Example session

`draft Thanks for your help` produces `DRAFT ONLY: You're welcome! Happy to help.` The user reviews it and decides whether to use it. `date` reports the local date at execution time; its output is intentionally not hard-coded in the book.

## What this teaches

The core function maps an input to a result. The conversation loop is separate. Rule-based matching has limitations: checking whether text contains `thank` is not real semantic understanding, and the first matching rule wins. Those are visible design choices, not hidden intelligence.

## A responsible extension architecture

**Input adapter → command router → approved tool or draft generator → preview → explicit confirmation → action.** Keep tool execution outside the text-generation function. A model's suggestion is untrusted input, not permission to run a shell command or send a message.

For voice input, request microphone permission and provide a visible stop control. Explain whether recordings leave the device. For external AI services, obtain consent before transmitting chat history, minimize the text sent, keep API keys in environment variables and handle network timeouts and rate limits. Never commit keys or private conversations to a repository.

Avoid brittle coordinate-based mouse automation as the foundation of a messaging tool. Screen layouts change, and a click in the wrong place may send text to the wrong person. Prefer supported APIs, platform rules and explicit user confirmation. Add deduplication and loop protection so two automated systems cannot endlessly reply to each other.

## Practice

Add a `time` command without changing `draft_reply()`. Then write tests showing that blank messages fail and that every draft is displayed for review. A future extension can replace only the draft-generation function while retaining confirmation controls.

# 29 · One Hundred Practice Challenges

Work in order within each group. For each challenge, record the input domain, an ordinary case, a boundary case and an invalid case when relevant. These are original study prompts, not a claim that every challenge has a full worked answer in this volume. Chapter 30 provides selected solutions; earlier lessons and projects solve related tasks.

## Foundations · 01–10

1. Print a three-line introduction with your learning goal.
2. Calculate a rectangle's area and perimeter for non-negative dimensions.
3. Convert a non-negative duration in seconds into hours, minutes and seconds.
4. Swap two variables without a temporary name; explain evaluation order.
5. Convert Celsius to Fahrenheit and back.
6. Find the final price after a validated discount percentage.
7. Compute simple interest and state the time unit used by the rate.
8. Convert a whole-number string to an integer, handling invalid text.
9. Format a decimal measurement to three places without claiming exact arithmetic.
10. Demonstrate why a phone number is usually stored as text.

## Decisions · 11–20

1. Determine whether an integer is even or odd.
2. Classify a number as positive, zero or negative.
3. Find the largest of three numbers without sorting.
4. Classify a valid mark using descending grade thresholds.
5. Test the Gregorian leap-year rule.
6. Determine whether three positive side lengths can form a triangle.
7. Build a menu that rejects unknown choices.
8. Check an age range with a chained comparison.
9. Normalize yes/no responses and reject unrelated input.
10. Test whether a character is a vowel under a clearly stated English-letter policy.

## Loops and numbers · 21–35

1. Sum integers from 1 through a non-negative `n`.
2. Calculate factorial iteratively; reject negative input.

3. Print a multiplication table with a chosen inclusive upper multiplier.
4. Determine whether an integer is prime; numbers below 2 are not prime.
5. List primes in an inclusive interval.
6. Produce the first `n` Fibonacci terms with a stated starting convention.
7. Compute the greatest common divisor with Euclid's algorithm.
8. Compute the least common multiple; define behavior when either input is zero.
9. Reverse the digits of an integer while preserving its sign.
10. Count digits in an integer, handling zero and a negative sign correctly.
11. Sum the decimal digits of an integer's absolute value.
12. Test whether a non-negative integer is an Armstrong number.
13. Test whether a positive integer is perfect.
14. Add values until the user enters a sentinel, without adding the sentinel.
15. Implement integer exponentiation for non-negative exponents.

### Strings · 36–45

1. Count whitespace-separated words.
2. Test a phrase palindrome using a stated normalization policy.
3. Count vowels and consonants among English alphabetic letters.
4. Build a character-frequency dictionary.
5. Find the most frequent character, breaking ties by first appearance.
6. Remove repeated characters while preserving first-seen order.
7. Determine whether two normalized phrases are anagrams.
8. Replace a target word without unexpectedly changing parts of longer words.
9. Find the longest word, defining how ties and punctuation are handled.
10. Validate a non-empty username against a simple documented rule.

### Lists, tuples, dictionaries and sets · 46–60

1. Calculate an average, rejecting empty input.
2. Find the second largest distinct value.
3. Remove duplicates while preserving input order.
4. Split a list into chunks; reject a non-positive chunk size.
5. Rotate a list by `k` positions; define the direction and handle an empty list.
6. Flatten a single level of nested lists.
7. Merge two sorted numeric lists into sorted output.
8. Build a 3×3 grid with independent rows.
9. Return minimum and maximum as an unpackable pair.

10. Count words with a dictionary.
11. Invert a dictionary while preserving all keys that share the same value.
12. Merge two dictionaries and state how collisions are resolved.
13. Compare two groups using set intersection and difference.
14. Find values appearing in exactly one of two sets.
15. Sort a collection of student records by score, then name.

### Functions and design · 61–70

1. Write a reusable positive-integer parser.
2. Write a function returning a Boolean rather than printing it.
3. Return quotient and remainder; reject a zero divisor.
4. Demonstrate a safe replacement for a mutable default argument.
5. Compare positional, keyword and default arguments in one small interface.
6. Use `*args` for a variadic numeric total without shadowing `sum`.
7. Write a closure that remembers a multiplier.
8. Write a decorator that preserves metadata with `functools.wraps`.
9. Compare iterative and recursive factorial and state their practical limits.
10. Split a text-analysis task into parsing, counting and reporting functions.

### Patterns · 71–80

1. Print a right-aligned star triangle.
2. Print an inverted left-aligned star triangle.
3. Print a centered filled pyramid.
4. Print a centered inverted pyramid.
5. Print a hollow square, including the size-1 case.
6. Print Floyd's number triangle.
7. Print a centered diamond without repeating its widest row.
8. Print a hollow centered pyramid.
9. Print a butterfly pattern with a clearly specified middle row.
10. Produce a numeric triangle whose counter continues between rows.

### Files, errors and tests · 81–90

1. Save and reload study goals as UTF-8 text.
2. Read several paths independently and report each missing file.
3. Write multiplication tables to a text file without overwriting unrelated data.
4. Read quoted CSV fields with the `csv` module.
5. Save a list of records to JSON and validate it on loading.

6. Recover from non-numeric input without a bare `except`.
7. Raise an exception when a function's domain is violated.
8. Test a calculation with `unittest`, including a boundary case.
9. Test file persistence inside a temporary directory.
10. Reproduce a bug with a minimal example, then add a regression test.

### Objects and projects · 91–100

1. Model a study session with independent per-instance state.
2. Create a validated property and demonstrate rejection of an invalid value.
3. Implement vector addition with appropriate unsupported-type behavior.
4. Compare composition and inheritance for a library borrowing system.
5. Add history to the calculator without mixing calculation and file I/O.
6. Load quiz questions from validated JSON data.
7. Add a monthly filter to the expense tracker.
8. Export expense records as CSV while retaining exact minor units internally.
9. Add a draft-preview confirmation step to an assistant without automatic sending.
10. Plan a capstone with a README, input contract, tests, sample data and a backup strategy.

# 30 · Selected Worked Solutions

These solutions demonstrate reusable reasoning. Attempt the relevant challenge first. Every snippet below is a standalone program in the companion examples. You can improve a solution after understanding its assumptions.

## A · Leap year · Challenge 15

A century year needs the stronger divisibility-by-400 rule. Otherwise the ordinary divisibility-by-4 rule applies.

```
def is_leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

for year in [1900, 2000, 2024, 2023]:
    print(year, is_leap_year(year))
```

**Output:** `1900 False`, `2000 True`, `2024 True`, `2023 False` on separate lines. Domain: positive Gregorian year numbers for this exercise.

## B · Prime test · Challenge 24

If `n` has a factor larger than its square root, the paired factor is smaller. Test possible divisors only through the integer square root.

```
from math import isqrt


def is_prime(number):
    if number < 2:
        return False
    for divisor in range(2, isqrt(number) + 1):
        if number % divisor == 0:
            return False
    return True

print([number for number in range(1, 21) if is_prime(number)])
```

**Output:** `[2, 3, 5, 7, 11, 13, 17, 19]`. Domain: integers. Checking 1 and a perfect square such as 49 catches common boundary errors.

## C · Fibonacci · Challenge 26

Choose the convention 0, 1, 1, 2, 3, … . Each update uses the previous pair of values simultaneously.

```
def fibonacci(count):
    if count < 0:
        raise ValueError("count cannot be negative")
    first, second = 0, 1
    result = []
    for _ in range(count):
        result.append(first)
        first, second = second, first + second
    return result

print(fibonacci(8))
print(fibonacci(0))
```

**Output:** `[0, 1, 1, 2, 3, 5, 8, 13]` and `[]`. Domain: non-negative integer count. Appending before the update ensures the first result is 0.

## D · GCD and LCM · Challenges 27–28

Euclid's algorithm replaces `(a, b)` with `(b, a % b)` until the remainder is zero. Dividing before multiplying keeps intermediate values smaller for LCM.

```
def gcd(a, b):
    a, b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a


def lcm(a, b):
    if a == 0 or b == 0:
        return 0
    return abs((a // gcd(a, b)) * b)

print(gcd(48, 18))
print(lcm(12, 18))
print(lcm(0, 18))
```

**Output:** `6`, `36`, `0`. Domain: integers. This convention gives `gcd(0, 0) == 0` and LCM zero whenever either input is zero.

## E · Signed digit reversal · Challenge 29

Handle the sign separately from the digits. Leading zeros disappear in the reversed integer: reversing 1200 produces 21, not a four-character string.

```
def reverse_integer(number):
    sign = -1 if number < 0 else 1
    remaining = abs(number)
    result = 0
    while remaining:
        result = result * 10 + remaining % 10
        remaining //= 10
    return sign * result

for number in [1200, -305, 0]:
    print(reverse_integer(number))
```

**Output:** `21`, `-503`, `0`. Domain: integers. Zero works because the initial result is already zero and the loop runs zero times.

## F · Normalized palindrome · Challenge 37

This policy ignores non-alphanumeric characters and compares caseless text. It is a simple Unicode-code-point exercise, not a full grapheme-aware language tool.

```
def is_phrase_palindrome(text):
    cleaned = "".join(character for character in text.casefold()
                      if character.isalnum())
    return bool(cleaned) and cleaned == cleaned[::-1]

print(is_phrase_palindrome("Never odd or even"))
print(is_phrase_palindrome("Python"))
print(is_phrase_palindrome("!!!"))
```

**Output:** `True`, `False`, `False`. Empty normalized text is deliberately rejected rather than labeled a palindrome.

## G · Stable frequency tie-breaking · Challenge 40

Count in first-appearance order. Python dictionaries preserve insertion order, and `max()` retains the first encountered item among equal maxima.

```
def most_frequent_character(text):
    if not text:
        raise ValueError("text cannot be empty")
    counts = {}
    for character in text:
        counts[character] = counts.get(character, 0) + 1
    return max(counts, key=counts.get)

print(most_frequent_character("baba"))
```

**Output:** `b`. Both letters occur twice, but `b` appeared first. Do not use `max(set(text), key=text.count)` when deterministic first-appearance ties matter.

## H · Chunking · Challenge 49

A positive step moves through start positions. Slicing naturally handles the shorter final chunk.

```
def chunks(values, size):
    if size <= 0:
        raise ValueError("size must be positive")
    return [values[start:start + size]
            for start in range(0, len(values), size)]

print(chunks([1, 2, 3, 4, 5], 2))
print(chunks([], 2))
```

**Output:** `[[1, 2], [3, 4], [5]]` and `[]`. Domain: a sliceable list and a positive integer size. Size zero and negative sizes are errors, not empty answers.

## I · Order-preserving deduplication · Challenge 48

A set answers “have I seen this?” while a list preserves the order of accepted items.

```
def unique_in_order(values):
    seen = set()
    result = []
    for value in values:
        if value not in seen:
            seen.add(value)
            result.append(value)
    return result

print(unique_in_order([3, 1, 3, 2, 1]))
```

**Output:** `[3, 1, 2]`. Domain: hashable values. For unhashable records, choose a stable key or use a different equality-based strategy.

## J · Safe mutable defaults · Challenge 64

Create a fresh list only when no list was supplied. An explicitly supplied list is deliberately modified; that behavior belongs in the contract.

```
def add_tag(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags

print(add_tag("python"))
print(add_tag("loops"))
```

**Output:** `['python']` and `['loops']`. The second call does not inherit the first call's tag.

## K · Closure and decorator · Challenges 67–68

The returned function remembers the multiplier in its enclosing scope. The decorator wraps a function while preserving useful metadata.

```
from functools import wraps


def multiplier(factor):
    def multiply(value):
        return value * factor
    return multiply


def announce(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"Calling {function.__name__}")
        return function(*args, **kwargs)
    return wrapper


@announce
def double(value):
    return multiplier(2)(value)

print(double(5))
```

**Output:** `Calling double` then `10`. `@announce` means that the name `double` now refers to the wrapper returned by `announce`.

## L · Floyd's triangle · Challenge 76

The number counter belongs outside both loops. The inner loop controls how many numbers go on this row, not where the counter restarts.

```
number = 1
for row in range(1, 5):
    values = []
    for _ in range(row):
        values.append(str(number))
        number += 1
    print(" ".join(values))
```

**Expected output**

```
1
2 3
4 5 6
7 8 9 10
```

## M · Diamond · Challenge 77

The lower half starts one row narrower than the widest row, avoiding an accidental double middle.

```
height = 3
rows = list(range(1, height + 1)) + list(range(height - 1, 0, -1))
for row in rows:
    print(" " * (height - row) + "*" * (2 * row - 1))
```

**Expected output**

```
  *
 ***
*****
 ***
  *
```

## N · Hollow pyramid · Challenge 78

The first and last star positions define the sloping sides; the final row fills the base.

```
height = 4
for row in range(1, height + 1):
    width = 2 * row - 1
    cells = []
    for column in range(width):
        border = column in (0, width - 1) or row == height
        cells.append("*" if border else " ")
    print(" " * (height - row) + "".join(cells))
```

**Expected output**

```
   *
  * *
 *   *
*******
```

## O · Vector addition · Challenge 93

A small dataclass communicates the fields. Returning `NotImplemented` for unrelated operands follows Python's operator protocol.

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Vector:
    x: float
    y: float

    def __add__(self, other):
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"


print(Vector(2, 3) + Vector(4, 1))
```

**Output:** `(6, 4)`. The annotations document numeric intent; they do not validate constructor inputs at runtime. The frozen dataclass prevents normal field reassignment, not arbitrary deep immutability for every possible field type.

# 31 · Quick Reference, Glossary and Next Steps

## The tools at a glance

| Need | First tool to consider |
|---|---|
| Read user text | `input()` |
| Display a result | `print()`, f-string |
| Choose a path | `if` / `elif` / `else` |
| Repeat until a condition changes | `while` |
| Visit items | `for` |
| Counter with items | `enumerate()` |
| Ordered editable sequence | `list` |
| Fixed sequence/group | `tuple` |
| Key-to-value lookup | `dict` |
| Uniqueness and membership | `set` |
| Reusable calculation | `def` and `return` |
| Paths and file locations | `pathlib.Path` |
| Structured local text data | `json` |
| Tabular text | `csv` |
| Expected failure recovery | Specific `except` clauses |
| Automated checks | `unittest` |
| Project dependencies | `venv`, `python -m pip` |

## Debugging checklist

- Did I save the correct file?
- Am I running the intended interpreter in the intended folder?

- What are the actual types and values at the failing line?
- Is the stop boundary inclusive or exclusive?
- Can the input be empty, zero, negative or duplicated?
- Am I returning a result or merely printing it?
- Am I unintentionally sharing a mutable object?
- Did I catch an error I should have allowed to reveal a bug?
- Does my output depend on set order or randomness?
- Can I reproduce the problem in fewer lines?

## Glossary

**Algorithm:** a step-by-step method for solving a defined problem.
**Argument:** a value supplied to a function call.
**Binding:** the association between a name and an object.
**Boolean:** a true/false value.
**Contract:** the expected inputs, outputs and failure behavior of an operation.
**Exception:** an object signaling that normal execution could not continue at a particular operation.
**Hashable:** suitable for hash-based lookup with consistent hash/equality behavior.
**Immutable:** not changeable in place at the object's own level.
**Instance:** a particular object created from a class.
**Iterable:** an object from which an iterator can obtain items.
**Iterator:** an object producing items one at a time until exhausted.
**Mutable:** changeable in place.
**Parameter:** a name declared in a function's definition for receiving an argument.
**Persistence:** retaining data beyond the life of a process.
**Recursion:** solving a problem through calls to smaller instances of the same problem.
**Scope:** the region/context in which a name is resolved.
**Trace:** a record of execution steps and changing values.
**Validation:** checking that data meets the requirements of an operation.

## A four-week practice route

**Week 1:** Chapters 1–8. Run each example, predict each output, and complete challenges 1–23. Keep a mistake notebook.

**Week 2:** Chapters 9–16. Draw patterns, manipulate text and collections, and rewrite repeated code as functions. Choose ten relevant challenges rather than racing through all of them.

**Week 3:** Chapters 17–24. Create a virtual environment, save validated data, read tracebacks and write tests. Build a tiny class only after explaining why it helps.

**Week 4:** Complete the projects one at a time. Add one feature, one boundary test and one README improvement to each. Finish with a capstone whose requirements fit on one page.

This is a flexible route, not a deadline. If a lesson is unclear, return to a smaller example and trace it. Understanding five lines deeply is more valuable than copying fifty lines you cannot explain.

## Trusted next reading

The Python documentation is the primary reference for language behavior and standard-library APIs:

- Python tutorial: https://docs.python.org/3/tutorial/
- Built-in functions: https://docs.python.org/3/library/functions.html
- Standard library: https://docs.python.org/3/library/
- Virtual environments: https://docs.python.org/3/library/venv.html
- Testing: https://docs.python.org/3/library/unittest.html
- Style guide, PEP 8: https://peps.python.org/pep-0008/

The uploaded beginner notes, program collections, pattern examples, input/output slides and CodeWithHarry handbook excerpts informed the selection of topics. The explanations, structure, worked examples and projects in this edition are newly written rather than reproduced screenshot pages. Those reference materials belong to their respective creators; they are not represented as original source publications by MaksDev Studio.

# Keep Building

You began with a line that displayed a result. You now have the ideas to turn a problem into a plan, a plan into code, and code into a program you can test and improve.

Do not wait until you know every feature. Pick one small problem that matters to you. Define its inputs. Write the simplest correct version. Test an awkward case. Explain your choices. Then build the next version.

**Understand the logic. Write the code. Build something useful.**

**Md. Makshedul Islam**
Student, EXIM Bank Agricultural University Bangladesh.

**MaksDev Studio**

- GitHub: MaksBloxX
- Repository: Python Project Lab
- LinkedIn: Md. Makshedul Islam